

Computer Graphics (CSED451-01)

Assignment 3.1: 3D Drawing with OpenGL Legacy Pipeline

Team Baguette - *Github Repository*

BLAIS Vladimir
CSED
49005916 - vblais

PICCAND Elliott
CSED
49005903 - piccandeliot

DEVELOPMENT ENVIRONMENT

To develop our program, we used VSCodium (with the clangd extension) for code editing, and CMake for compiling. However, instruction to compile with Visual Studio are available in the project README.md. To understand the source code, the reader must be familiar with modern C++ features (C++ 23).

In addition, we used the [Tracy](#) library to profile our game when we faced performances issues. This library was only used during the development and is not needed when compiling the game in Debug or Release mode.

PROGRAM DESIGN AND IMPLEMENTATION

Our program contains several features, including:

- basic player controls;
- cannon balls;
- collisions;
- radar;
- enemies;
- damage system;
- victory / defeat display;
- multiple views;
- explosion effects;
- foam trails;
- cannon recoil;
- flapping flag;
- water splash;
- sparks on cannon balls.

and some features not visible by the players, but useful for development:

- hierarchical system;
- component system;
- event system;
- input system;
- resource loader;
- physics system;
- particle system.

In order to compile our program, there are some additional requirements¹:

- enabling C++23 features. This choice was made to be able to use the C++23 `std::ranges` and `std::views` features, as well

as the `<print>` header, to shorten development time and code readability.

- the stb (single-file public domain (or MIT licensed) libraries for C/C++) library. This allow us to load images for the game (we only use the `stb_image.h` header).
- the TinyGLTF library. This allow us to load glTF 3d models.

To summarize how our program works, we can use the following pseudo-code (see `alg:main-loop`)

Algorithm 1. – Game Main Loop

```
1: function MAIN-LOOP()
2:   while  $\neg$  Window-Should-Close() do
3:     UPDATE-INPUT-SYSTEM()
4:     PROCESS-EVENTS()
5:     ▷ update the different elements of the game, recur-
       sively from the scene root
6:     UPDATE-SCENE()
7:     UPDATE-PHYSICS-SYSTEM()
8:     UPDATE-PARTICLE-SYSTEM()
9:
10:    ▷ render everything (this is a constant function, no
       update occurs here)
11:    RENDER-SCENE()
12:    RENDER-PARTICLES()
13:
14:    ▷ display the rendered content on the screen
15:    SWAP-FRAMEBUFFERS()
16:  end
17: end
```

This is a usual game main loop. Sometime, games place the `Process-Events()` part at the end of the loop, but we decided to place it at the beginning, since it has no impact on the behavior of the program, and allow to defer calls during the initialization (even if we do not need that so far, we might need it in the future).

Technical Features

Hierarchical System : We defined our scene as a tree of `GameObject`. By default, each game object only handle its relation with its parent/children and do nothing particular beside propagating the `update()` and `render()` methods.

Component System : Each `GameObject` have a list of components attached to it. Each component is a small brick of behavior. Here are some of the main components:

- **Transform**: handle the position, orientation and scale of a `GameObject` relatively to its parent;

¹Please check the README.md for detailed instructions.

- **Collider**: represent a region of space. Allow collision detection;
- **RigidBody**: represent an element of the Physics engine. Allow movement and collision solving;
- **ModelInstance**: attach a `Model` to the `GameObject`.

Event System : To allow every part of our code to interact between each other, the event system allow to register callbacks and post events. Those events are then processed at the same time as the window events.

Input System : This is a simple abstraction layer that allow developers to get keyboard and mouse input without having to deal with glfw heavy syntax and duplicate bindings.

Resource Loader : This is a wrapping around a hash map that only load each 3d model / texture once, even if it is used at multiple places in the code base. For instance, since each ship's model has a lot of polygons, we do not want to load the model for each player and enemy. Instead, we use the resource loader to load the model once in memory, and then use references to this memory to draw each ship.

Physics System : This system allow to apply forces to each rigid body in the scene. Each rigid body is then moved and rotated accordingly to Newton's laws. In addition, this part also handle collision detection for triggers, and collision solving when needed.

Particle System : This system is used extensively in the game to render huge amount of colored points. It handle the display part but also the motion of each particle.

Visual Features

Player Ship : The player has a ship (see Fig. 1) that it can control.

Cannon : Each ship has a cannon (see Fig. 2 & Fig. 3) that is made of 2 parts: the stand that rotate toward the target, and the barrel that move up/down depending on how far the target is. The maths used to compute the cannon inclination can be found inside [Ballistic.pdf](#).

Cannon Balls : Each cannon can fire cannon balls (see Fig. 4). Cannon balls follow a parabolic trajectory (see Fig. 5 & Fig. 12) matching Newton's laws thanks to the physics engine.

Collisions : The game use a lot of collision, for triggers like when a cannon ball hit a ship and also for interaction between ships and walls. Different types are implemented:

- **Axis Aligned Bounding Box (AABB)**: this collider is used mostly as an optimization, for pre-checking if a collision occurs between two collider (see Fig. 6). However, some simple objects such as the cannon balls have only this type of collider and no more precise one.
- **Convex Polyhedron**: This is the main collider used in the game for the ship (see Fig. 7). Its a low resolution mesh, edited to be convex.

- **Sphere**: We mainly used this type of collider for explosions damage area (see Fig. 8).

Also, because we did not wanted the ships and cannon balls to hit an invisible wall when reaching the border of the map, we added some rocks all around the ocean (see Fig. 9 & Fig. 11). Those rocks are purely visual and the collisions are handle by an AABB collider on each map edge (see Fig. 10 & Fig. 12).

Radar : Each ship feature a rotating radar made of a brown cylinder and a red cone (see Fig. 2 & Fig. 13).

Enemies : When starting, the game spawn some enemies (see Fig. 14). Each enemy motion follow this process: the enemy pick a random point of the map (not too close to the current ship position), then rotate and move in a straight line toward this waypoint. Once reached, another waypoint is selected and the process restart. We also use the same process for the enemies' cannon's target: each target has its own waypoint, and the target move toward the waypoint. Those waypoints can be seen in debug mode: the waypoint itself is a 1m side red cube on the water, and its link to the ship or the ship's target by a green dashed line (see Fig. 12).

Damage System : Each ship collider detects if a cannon ball enter collides with it. If this event occurs, the cannon ball is destroyed and is replaced by an explosion. This explosion has its own collider. If it touch the ship's one, the ship takes randomly between 3k and 15k damages. Each ship has 24k hit points. If the hit points reach 0, the ship disappear.

Health bars are displayed over each ship (see Fig. 11). To render them, we use a simple trick: instead of implementing an entire UI system, we simply drew 2 quads high in the sky, and by enabling back face culling, those are only visible from the top view, and not from the cannon/cannon ball view (see Fig. 15).

Victory / Defeat Display : On victory or defeat, we used the same tricks as for the health bar to render a simple hand drawn texture with the texts (see Fig. 16 & Fig. 17 & Fig. 15).

Multiple Views : The game feature 3 view modes:

- The Top view (see Fig. 11);
- The Free View (for debug purposes, toggled by clicking Enter);
- The Cannon/Cannon ball view: if at least one cannon ball fired by the player is still flying, the camera follow the last fired one (see Fig. 19). Otherwise, it is located near the player's cannon, facing toward the target (see Fig. 18).

Explosion Effects : When a cannon ball hit anything but water, it spawn an explosion (see Fig. 20). To render it, we used 2500 particles with different colors and velocity.

Foam Trails : When a ship is moving, it spawn behind it a foam trail (see Fig. 21). To render it, we used 50 particles per frame with different position, velocity and lifetime, subject to gravity.

Cannon Recoil : Whenever a canon fires, the barrel have a recoil effect. However, because the views instantly swap to the cannon ball's (because of the assignment requirements), this is only visible in cannon/cannon ball view when enemies are close enough, or in free camera view.

Flapping Flag : Each ship has its topmost flag (see Fig. 2) flapping. However, since the health bar is above the ship, this is only visible in cannon/cannon ball view when enemies are close enough, or in free camera view.

Water Splash : When a cannon ball hit the water plane, it spawn an water splash (see Fig. 22). To render it, we used 500 particles with different colors, position, velocity and lifetime, subject to gravity.

Sparks on Cannon Balls : Each cannon ball has animated sparks on its stem (see Fig. 4). This is done by spawning a few particle each frames. This is also that which allow the player to see the cannon balls in top view. Otherwise, they would be too small and only appear as a 1 pixel wide black dot.

Debug Mode : Not really a player-oriented feature, but we added a debug mode to the game (it can be toggled by clicking F3) to display useful debug information such as:

- colliders (see Fig. 6 & Fig. 7 & Fig. 8 & Fig. 10);
- waypoints (see Fig. 12);
- cannon ball estimated trajectories (see Fig. 5 & Fig. 12);
- game's and model's axis (see Fig. 23). This helped a lot to debug model orientation to see their scale and orientation;
- perspective cameras (see Fig. 24).

END-USER GUIDE

The game starts immediately on running the executable. The goal is to sunk the enemies' ships (the ones with black sails).

The player (ship with white sails) can change its ship speed with the W and S keys (respectively increasing and decreasing the ship speed), and rotate it using the A and D keys (turning the ship respectively left and right by 15°). The player can also switch between 2 views by clicking (press and release) V:

- Top view: in this view, the player sees the entire world, and can use its mouse to fire cannon balls. Holding pressed the Left Mouse Button enable aiming mode. In aiming mode, the player can either fire and exit the aiming mode (by releasing the Left Mouse Button), or exit without firing by clicking (pressing then releasing) the Right Mouse Button.
- Cannon / Cannon ball view: in this view, the player can no longer fire, but can still move its ship.

When a cannon ball hit a ship, it deals randomly between 3000 and 15000 damages. Each ship has 24000 hit points. When a ship reach 0 or less hit points, it sink. The player win if every enemies are sunk, and lose if its ship get sunk.

Additionally, at any time, the player can:

- click ESC to quit the game
- click G to restart the game
- click F11 to toggle fullscreen

Debug view and controls

To help with debugging, a few debug options are available :

- clicking R cycle between the different rendering mode. For now, the available rendering modes are:
 - Default
 - Wireframe
- clicking Enter toggle a free view camera that can be moved with W A S D Space Left Shift and moving the mouse. (note that here, W A S D no longer move the ship).
- clicking P toggle on/off the physics and the particle engines (not the animations tho)
- clicking F when in another view than Top View fire a cannon ball from the player ship
- the arrow keys Up Left Down Right move the player's cannon's target respectively North, West, South and East.
- clicking F3 toggle the display of a lot of debug information such as :
 - colliders: AABB with red boxes, Convex Polyhedron with green wireframe and Sphere with cyan circles
 - cannon's predicted trajectories: blue line
 - ship's targets: represented by a 1x1x1 aabb collider at the end of the blue line
 - enemy ships' waypoints: represented by a 1x1x1 aabb collider. A green dashed line goes from the ship to this waypoint
 - enemy ships' targets' targets: same as the enemy ships' waypoint, but starting from the current ship's target.

Game Screenshots



Fig. 1. – Player Ship

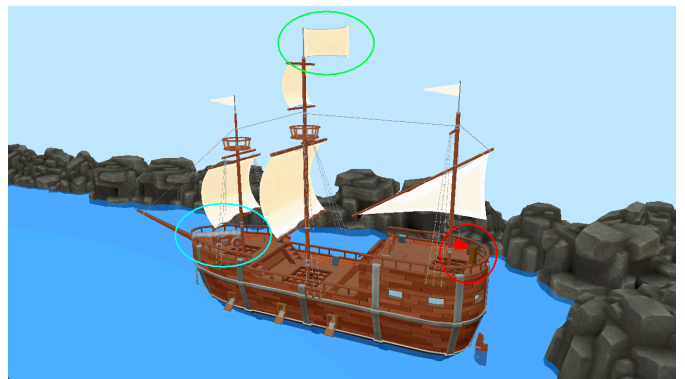


Fig. 2. – Player Ship with its radar (red), its flapping flag (green) and its cannon (cyan)

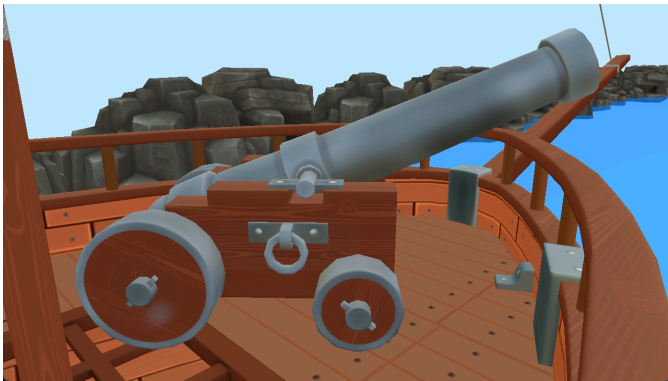


Fig. 3. – Cannon stand (wooden part and wheels) and barrel (metallic part)

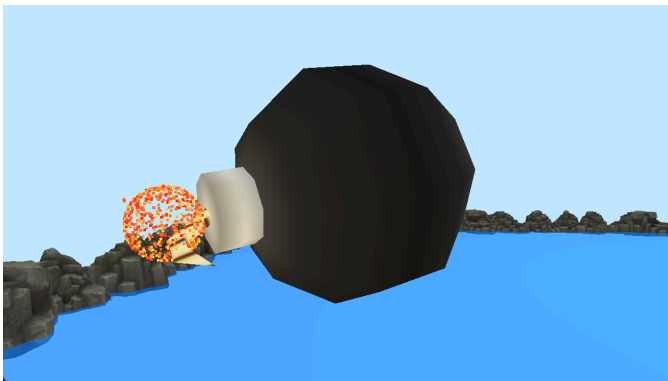


Fig. 4. – Cannon ball and sparks on its stem²

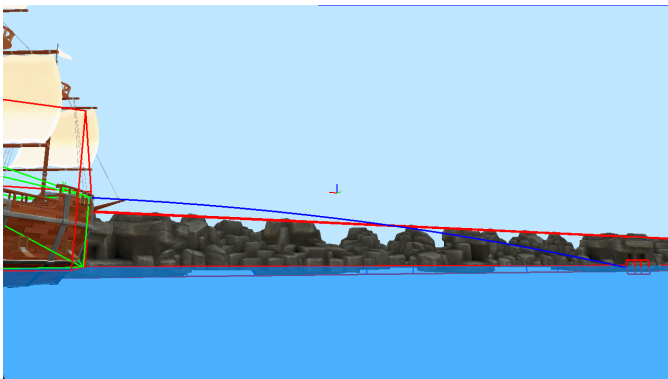


Fig. 5. – Cannon ball trajectory preview (blue line) visible with the debug mode (F3) enabled

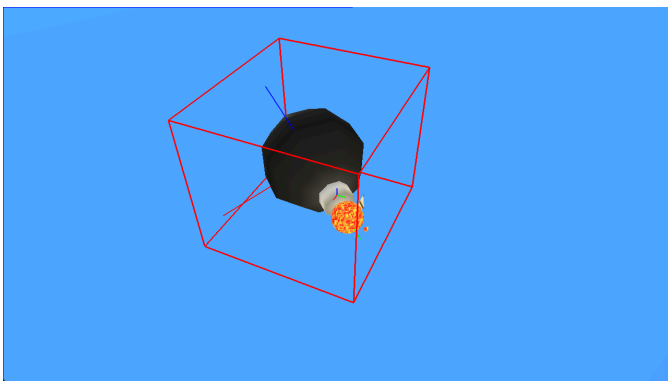


Fig. 6. – AABB collider (in red) around a cannon ball

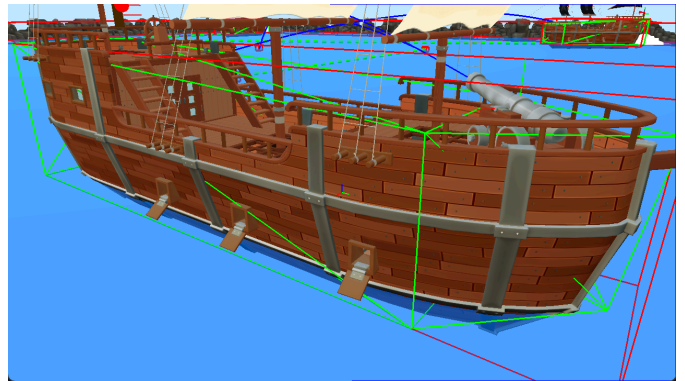


Fig. 7. – Convex polyhedron collider (in green) and its AABB (in red) around a ship

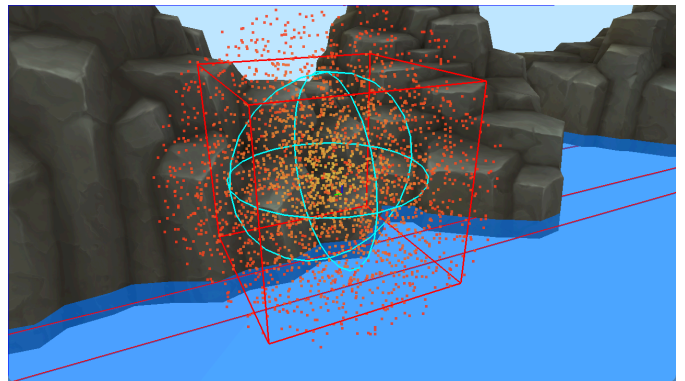


Fig. 8. – Sphere collider (in cyan) and its AABB (in red) inside an explosion

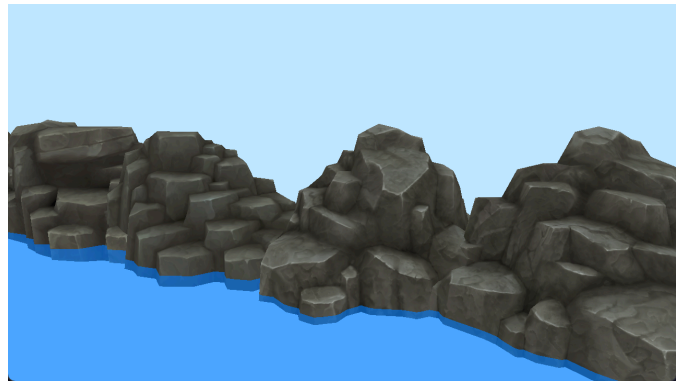


Fig. 9. – Rocks at the edge of the map

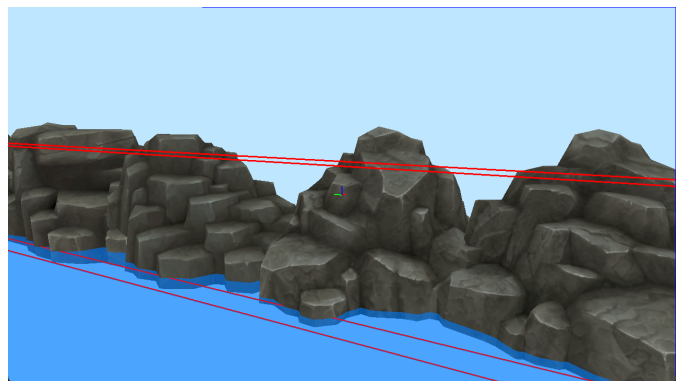


Fig. 10. – Rocks at the edge of the map and the map edge collider

²The amount of spark's particles on the screenshot is bigger than the one seen in game because of the time needed to take the screenshot.

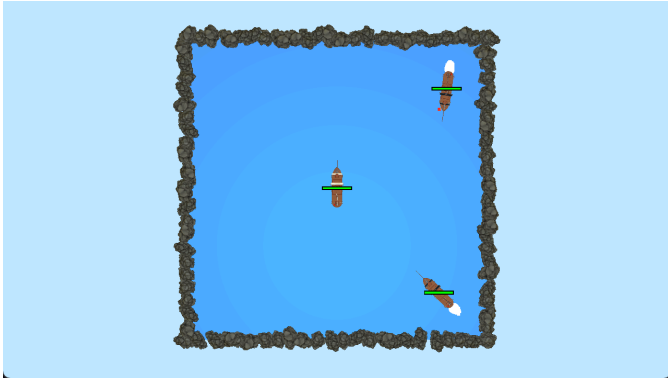


Fig. 11. – Top view at the start of the game

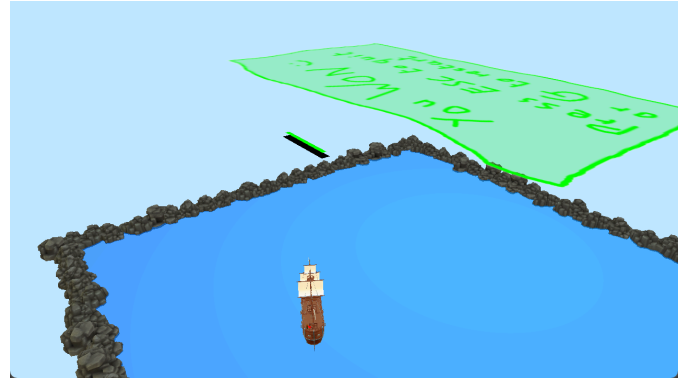


Fig. 15. – Health bar and victory menu viewed from the free view camera

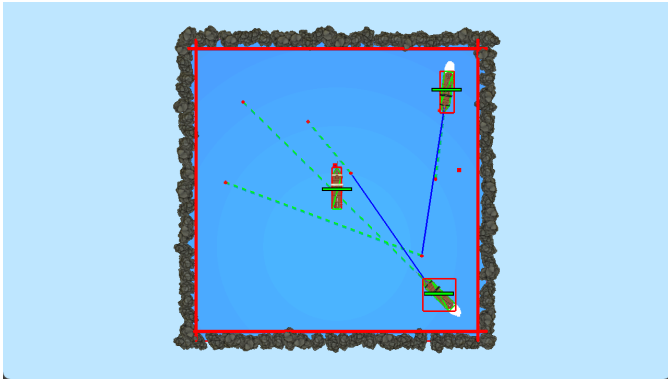


Fig. 12. – Top view at the start of the game with debug mode enabled



Fig. 16. – Victory screen

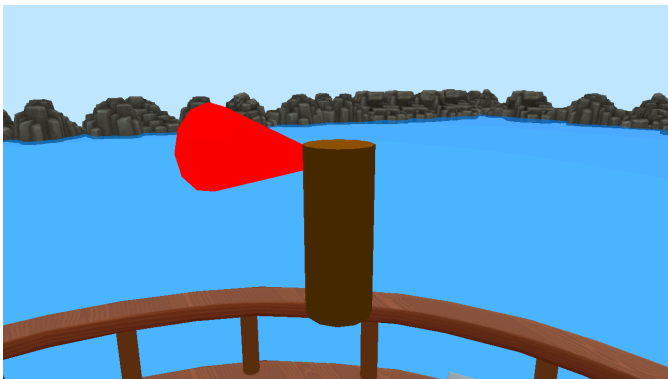


Fig. 13. – Radar at the back of each ships



Fig. 17. – Defeat screen



Fig. 14. – Enemy ship³

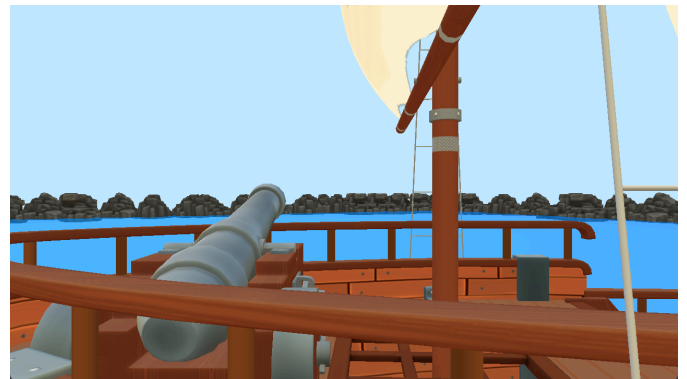


Fig. 18. – Cannon view

³The player and enemies ships share the same 3d model. Only the sails texture is updated on the player to have a different look. Thus, only one model is loaded into the memory.

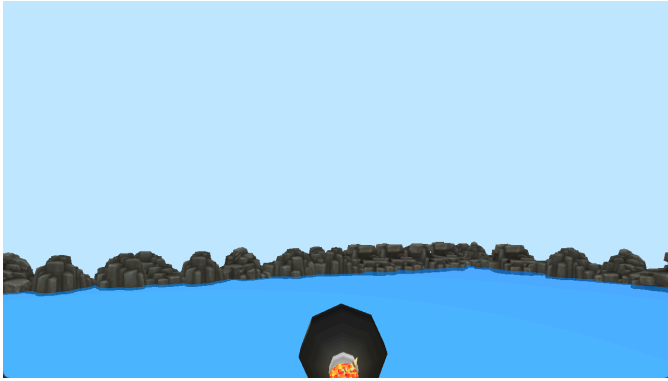


Fig. 19. – Cannon ball view

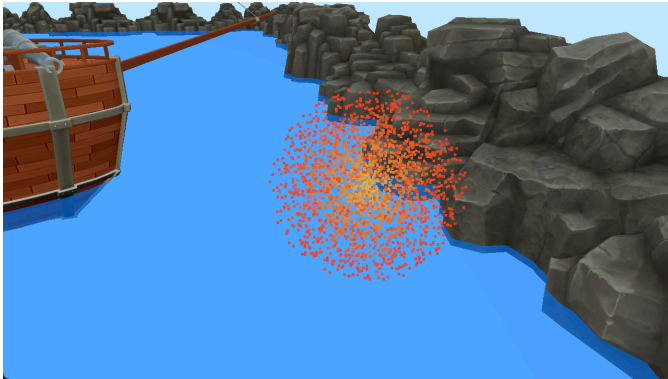


Fig. 20. – Explosion



Fig. 21. – Foam trail



Fig. 22. – Water splash

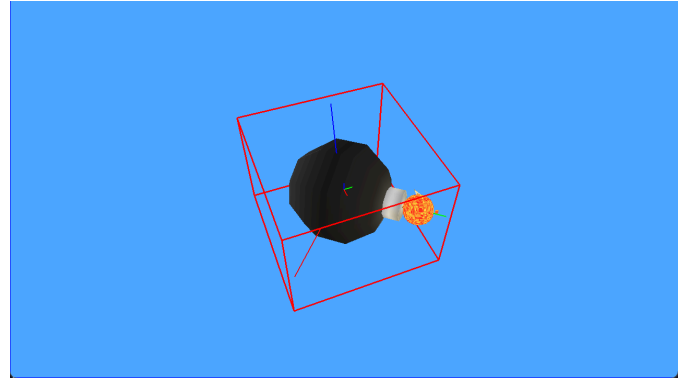


Fig. 23. – Debug view of a cannon ball with on the middle of the screen, the game's axis and on the cannon ball, its model axis (red = X, green = Y and blue = Z)

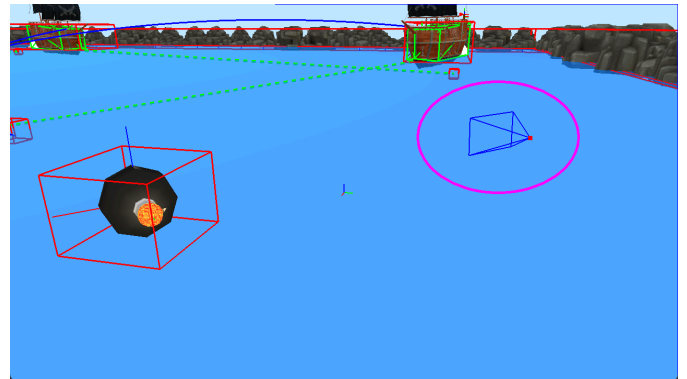


Fig. 24. – Debug view of the camera following the cannon ball

DISCUSSIONS/CONCLUSIONS

During the development, we didn't encounter much issues. We just spent a lot of time finding good assets for the game and solving orientation issues with those models.

REFERENCES

Every part of the code is original, but we use several tutorials or references during the development :

- the camera shaking part mechanic is greatly inspired by [miklatov answer on this Stack Exchange discussion](#);
- the component system was inspired by [this Vulkan tutorial](#).

AI-ASSISTED CODING REFERENCES

During the development, we used generative AI when some part require learning a lot of non-graphical related topic to implement. So we always designed everything without AI, but sometime use AI to implement some designs such as :

- Copilot helped implementing the collision solving system (not detection);
- Copilot helped implementing the glTF model loading.

Thus, we estimate that 8% of the code was generated by an LLM.